**Course Code: CSA1741** **Course Name: ARTIFICIAL INTELLIGENCE**

**Slot: B**

**ASSIGNMENT 2 – CO2**

**Nature of Assignment: Individual** **Duration: 1 Week** **Total Marks: 50**

**Student Name: VISHAL.V** **Reg. No: 192419033**

**CO2 Coverage:** Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimization problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

**TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)**

**Problem Context – Autonomous Campus Shuttle**

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.

**Your Challenge**

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

- Predict how the three algorithms will behave under different map conditions
- Explain the algorithmic reason for your prediction
- Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Nodes expanded and nodes generated
- Maximum frontier size
- Path cost and whether the path is optimal
- Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

**Design Decision**

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

**Evidence Expected:** Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

**TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)**

**Problem Context – Exploration Rover in an Unmapped Facility**

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

**Your Challenge**

Design experiments to investigate the online agent under:

- Targets at different distances from the start
- Unreachable targets
- Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

**Changed Condition**

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

**Evidence Expected:** Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

**TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)**

**Problem Context – Game Engine for a Board Game**

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

**Your Challenge**

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs)
2. One game tree deliberately engineered so that almost no cutoffs occur
3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

- Predict the expected number of nodes visited
- Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs.

**Inquiry**

Compare prediction with experimental result. Explain:

- Why one tree requires substantially more work than another
- Which characteristics of move ordering create or prevent cutoffs
- What this reveals about evaluating an algorithm using only the branching factor and depth

**Evidence Expected:** Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

**TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)**

**Problem Context – Turn-Based Strategy Game**

A game AI must choose its next move by looking ahead in a game with an average of b legal moves per turn.

**Your Challenge**

Design a depth-limited minimax solution with an evaluation function of your own.

Before implementation, determine:

- How many positions are examined for lookahead depth d
- How the evaluation function is used when the search is cut off
- How the number of terminal evaluations changes with d

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

- Number of nodes expanded and evaluation-function calls
- Execution time, where useful
- Quality of the move chosen (for example, win rate over 20 games against a random player)

Compare the theoretical growth with experimental behaviour.

**Scalability Inquiry**

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

**Evidence Expected:** Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

**TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)**

**Problem Context – University Exam Timetabling**

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

**Design Starting Point**

You are given only the following principle:

A value may be removed from a variable's domain if some neighboring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

**Your Challenge**

Convert this principle into a computational procedure. Determine:

- How the constraints are represented
- How a value without support is detected
- Which other pairs must be re-examined after a value is removed
- What condition indicates that the problem has no solution
- How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

**Test Your Design**

Test the algorithm using:

- A timetable instance in which propagation solves or detects failure without any search
- A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

**Correctness Inquiry**

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

**Evidence Expected:** A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

**TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)**

**Problem Context – Delivery Tour Planning**

A courier must visit n customers exactly once and return to the depot, minimising the total travel distance.

**Your Challenge**

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

**Correctness Check:** Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

**Growth Investigation**

Before experimentation, derive/predict how the number of candidate tours grows with n. Use at least three problem sizes to validate your prediction experimentally.

**Local Search Alternative**

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

**Inquiry**

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

**Deployment Decision**

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest n for which you would deploy exhaustive search, and the point at which you would switch to local search.

**Evidence Expected:** Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

**DAY-4 DESIGN CHALLENGE**

**TASK 7 – Diagnose, Redesign and Validate (13 Marks)**

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

**A. Diagnose the Computational Bottleneck – 2 Marks**

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

**B. Define the Design Objective – 2 Marks**

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

**C. Design the Revised Computational Strategy – 4 Marks**

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

**D. Predict Before Implementing – 1 Mark**

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

**E. Implement and Validate the Redesign – 2 Marks**

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

**F. Defend the Final Design – 2 Marks**

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

**MARK DISTRIBUTION – ASSIGNMENT 2**

| Task | Assessed Area | Marks |
|---|---|---|
| Task 1 | Uniform-cost, Greedy and A* search | 6 |
| Task 2 | Online search agent | 4 |
| Task 3 | Alpha-beta pruning | 6 |
| Task 4 | Minimax and evaluation functions | 6 |
| Task 5 | Constraint propagation and CSP | 7 |
| Task 6 | Exhaustive and local search, optimisation | 8 |
| Task 7 | Day-4 Design Challenge – Diagnose, Redesign & Validate | 13 |
| | **TOTAL** | **50** |

**Submission Requirements**

Submit:

- Your assignment report containing the required evidence for Tasks 1–7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

**Assessment Verification**

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

**Overall Assessment Expectation**

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

**Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate**

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

# SLOUTION: -

## TASK 1 – Greedy Best-First Search and A* Under Different Map Conditions

### 1.1 Introduction

An autonomous campus shuttle needs to find the shortest route between two locations. However, some roads may be blocked due to construction or other problems.

To solve this, we can use three search algorithms:

1. Uniform Cost Search (UCS): Finds the path with the lowest total cost.
2. Greedy Best-First Search: Selects the location that appears closest to the destination.
3. A Search:* Combines the actual cost and estimated remaining distance to find an efficient path.

### 1.2 Prediction of Algorithm Behaviour

| Algorithm | Working principle | Expected behaviour |
|---|---|---|
| UCS | Selects the lowest path cost | Finds the optimal path but may explore more nodes |
| Greedy Best-First | Selects the smallest heuristic value | Usually faster but may select a longer path |
| A* | Uses actual cost + estimated cost | Efficient and finds an optimal path with an admissible heuristic |

The A* evaluation function is:

$$f(n) = g(n) + h(n)$$

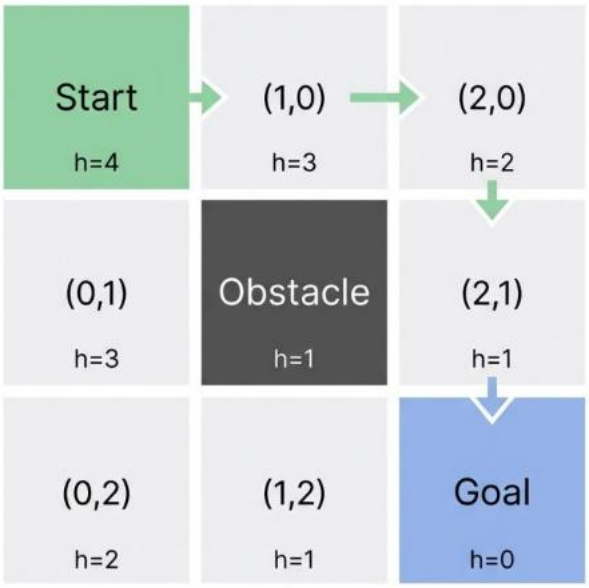
Where:

- g(n) = Cost from the starting point.
- h(n) = Estimated distance to the destination.
- f(n) = Total estimated cost.

### 1.3 Designing Different Maps

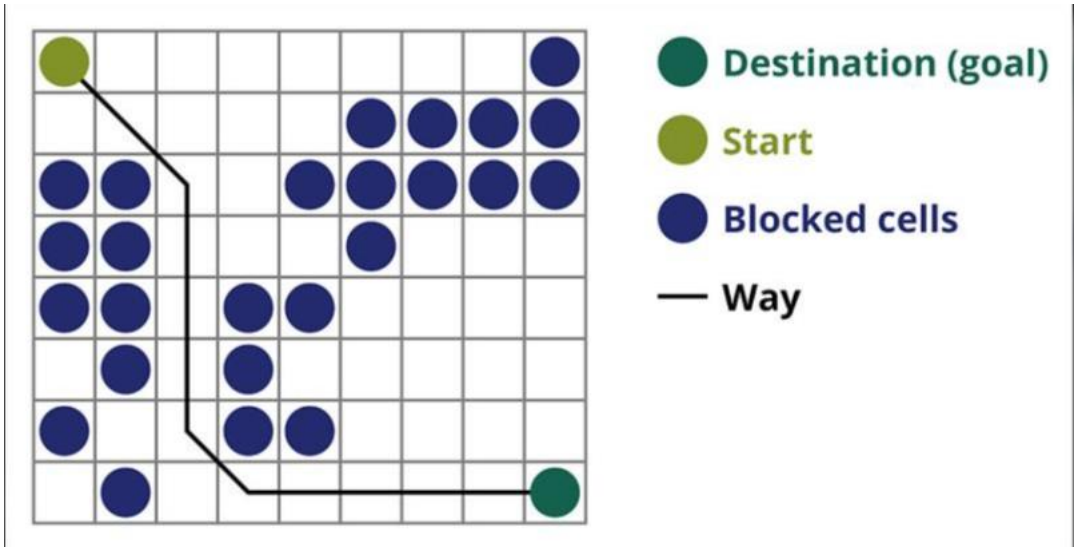
I have designed three different map conditions to compare the algorithms.

**Map 1 – Open Map**



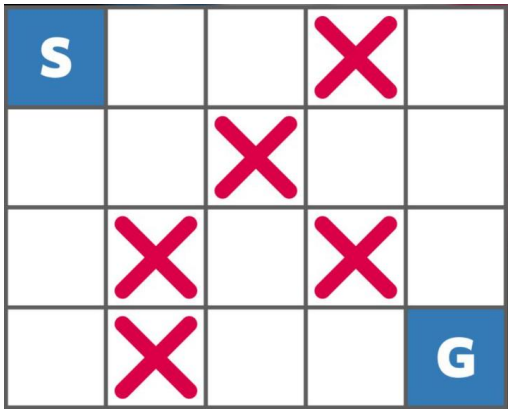

- Very few obstacles.
- Most roads are directly accessible.
- All three algorithms should find a route easily.
- A* is expected to explore fewer unnecessary locations.

## Map 2 – Cluttered Map




- Several obstacles block the direct route.
- Greedy search may get attracted towards a blocked or misleading direction.
- UCS explores according to actual path cost.
- A* balances actual cost and estimated distance.

## Map 3 – Unreachable Goal




- The destination is completely surrounded by obstacles.
- All algorithms should eventually report that no path exists.
- This tests whether the algorithms correctly handle failure cases.

### 1.4 Admissible Heuristic Function:

For our grid map, I use Manhattan Distance.

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

It calculates the minimum number of horizontal and vertical movements needed to reach the destination.

It is admissible because it never overestimates the actual travel cost when every movement costs 1.

### 1.5 Python Program

The following program implements all three algorithms and records the number of expanded nodes.

```
import heapq

def search(grid, start, goal, method):
    rows, cols = len(grid), len(grid[0])
    def heuristic(p):
        return abs(p[0]-goal[0]) + abs(p[1]-goal[1])
    queue = [(heuristic(start) if method == "Greedy" else 0,
              0, start, [start])]
    visited = set()
    expanded = 0
    while queue:
        priority, cost, node, path = heapq.heappop(queue)
        if node in visited:
            continue
```

```
        visited.add(node)
        expanded += 1
        if node == goal:
            return path, cost, expanded
        x, y = node
        for dx, dy in [(0,1), (1,0), (0,-1), (-1,0)]:
            nx, ny = x+dx, y+dy
            nxt = (nx, ny)
            if (0 <= nx < rows and 0 <= ny < cols
                and grid[nx][ny] != 1
                and nxt not in visited):
                new_cost = cost + 1
                if method == "UCS":
                    score = new_cost
                elif method == "Greedy":
                    score = heuristic(nxt)
                else:
                    score = new_cost + heuristic(nxt)
                heapq.heappush(
                    queue, (score, new_cost, nxt, path+[nxt])
                )
    return None, -1, expanded
grid = [
```

```
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)
for method in ["UCS", "Greedy", "A*"]:
    path, cost, nodes = search(grid, start, goal, method)
    print("\nAlgorithm:", method)
    print("Path:", path)
    print("Cost:", cost)
    print("Expanded Nodes:", nodes)
```

### 1.6 Comparison and Analysis

| Condition | UCS | Greedy | A* |
|---|---|---|---|
| Open map | Good | Fast | Very good |
| Cluttered map | Optimal | May take longer route | Optimal |
| Unreachable goal | Reports failure | Reports failure | Reports failure |

The actual number of expanded nodes depends on the map and tie-breaking order.

### 1.7 Final Design Decision

The operations team reports that 95% of shuttle trips happen on open roads with only occasional detours.

I recommend A Search* for the shuttle system because:

- It considers both actual travel cost and estimated distance.
- It finds an optimal path when the heuristic is admissible.
- It can handle obstacles and unexpected detours.
- It avoids much unnecessary exploration compared with UCS in many maps.

Although Greedy Search can be faster in some situations, it does not guarantee the shortest path.

Conclusion: A* is the most balanced choice because it provides both efficiency and optimal path selection.

# TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice?

## 2.1 Introduction

An online search agent is an intelligent agent that explores an unknown environment while performing its task.

For example, an exploration rover inside an unmapped building does not know where the charging station is. It can only observe the nearby cells and must learn the map while moving.

Unlike normal search, online search does not require complete knowledge of the environment in advance.

## 2.2 Working of Online DFS

Online Depth-First Search (DFS) explores one direction at a time.

Its working steps are:

1. Start from the initial location.
2. Observe the nearby cells.
3. Select an unvisited and accessible cell.
4. Move to that cell.
5. Store the previous location.
6. If a dead end is reached, return to the previous location.
7. Continue until the target is found.

### 2.3 Python Program – Online DFS

```
def online_dfs(start, goal, grid):
    visited = set()
    path = []
    moves = [(0,1), (1,0), (0,-1), (-1,0)]
    def dfs(pos):
        if pos == goal:
            path.append(pos)
            return True
        visited.add(pos)
        path.append(pos)
        x, y = pos
        for dx, dy in moves:
            nxt = (x+dx, y+dy)
            if (0 <= nxt[0] < len(grid)
                and 0 <= nxt[1] < len(grid[0])
                and grid[nxt[0]][nxt[1]] == 0
                and nxt not in visited):
                if dfs(nxt):
                    return True
        path.pop()
        return False
```

```
    if dfs(start):
        return path
    return None
grid = [
    [0, 0, 1, 0],
    [1, 0, 1, 0],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]
```

print("Path:", online_dfs((0,0), (3,3), grid))Note: This is a simplified DFS demonstration using a supplied grid. A real online agent would discover neighbouring cells through sensors rather than receive the complete grid.

### 2.4 Experimental Conditions

| Test condition | Expected behaviour |
|---|---|
| Nearby target | Reaches the target quickly |
| Distant target | Requires more movements |
| Unreachable target | Explores available paths and reports failure |
| Small maze | Works well |
| Large maze with dead ends | May revisit many locations |

### 2.5 Online DFS vs LRTA*

LRTA* (Learning Real-Time A*) uses local information and updates its estimated cost after every move.

| Online DFS | LRTA* |
|---|---|
| Uses depth-first exploration | Uses heuristic values |
| May explore many dead ends | Tries to move towards promising locations |
| Uses a stack | Updates estimated costs |
| Simple to implement | Requires more computation |

### 2.6 When Should We Use Online Search?

Online search is useful when:

- The map is not available in advance.
- The environment changes frequently.
- The robot can sense nearby locations.
- The robot must make decisions while moving.

However, when the map is already known, online search is less useful because it may waste time exploring areas that could have been avoided.

## 2.7 Changed Condition: 20 Rovers in 20 Facilities

If 20 rovers are deployed in 20 facilities and their maps remain unchanged for months, I would change my recommendation.

Instead of exploring the entire building repeatedly, the system should:

1. Explore and map each facility initially.
2. Save the discovered map.
3. Calculate efficient routes using the stored map.
4. Reuse the route information for future missions.
5. Update the map only when changes are detected.

Conclusion: Online search is suitable for one-time exploration of unknown environments. For repeated missions in an unchanged environment, offline planning using a stored map is more efficient.

## TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning

### 3.1 Introduction

Minimax is an algorithm used in two-player games. It helps a player choose the best possible move by considering the opponent's possible responses.

- MAX player tries to get the highest score.
- MIN player tries to get the lowest score.

Alpha-Beta Pruning improves Minimax by avoiding branches that cannot change the final decision.

It produces the same result as Minimax but can examine fewer nodes.

### 3.2 Working of Alpha-Beta Pruning

It uses two values:

- Alpha (α): The best value currently available to MAX.
- Beta (β): The best value currently available to MIN.

When:

$$\alpha \geq \beta$$

The remaining branches can be skipped because they cannot affect the final decision.

### 3.3 Three Test Conditions

I designed three different conditions to understand how pruning behaves.

Test 1 – Good Move Ordering

- The best moves are checked first.
- Alpha-Beta can identify unnecessary branches early.
- More nodes can be pruned.

Test 2 – Poor Move Ordering

- The best moves are checked last.
- The algorithm takes longer to identify useful cutoffs.
- More nodes are examined.

Test 3 – Equal or Random Values

- Leaf nodes contain equal or randomly arranged values.
- Pruning depends on the order in which the values are checked.
- The number of visited nodes may change.

### 3.4 Python Program

```
def minimax(tree, is_max):
    if isinstance(tree, int):
        return tree
    values = [minimax(x, not is_max) for x in tree]
    if is_max:
        return max(values)
    return min(values)
def alpha_beta(tree, alpha, beta, is_max, count):
    count[0] += 1
    if isinstance(tree, int):
        return tree
    if is_max:
        best = float('-inf')
        for child in tree:
```

```
            best = max(best, alpha_beta(
                child, alpha, beta, False, count
            ))
            alpha = max(alpha, best)
            if alpha >= beta:
                break
        return best
    else:
        best = float('inf')
        for child in tree:
            best = min(best, alpha_beta(
                child, alpha, beta, True, count
            ))
            beta = min(beta, best)
            if alpha >= beta:
                break
        return best
tree = [
    [[3, 5], [6, 9]],
    [[1, 2], [7, 8]]
]
print("Minimax Result:", minimax(tree, True))
count = [0]
```

```
result = alpha_beta(
    tree, float('-inf'), float('inf'), True, count
)
print("Alpha-Beta Result:", result)
print("Nodes Visited:", count[0])
```

### 3.5 Comparison

| Method | Nodes examined | Performance |
|---|---|---|
| Minimax | All required nodes | More computation |
| Alpha-Beta (good ordering) | Fewer nodes | Faster |
| Alpha-Beta (poor ordering) | More nodes | Slower |
| Alpha-Beta (equal values) | Depends on ordering | Variable |

### 3.6 Analysis

The performance of Alpha-Beta Pruning mainly depends on the order of moves.

When good moves are checked first, the algorithm gets better alpha and beta values early. This helps it remove unnecessary branches.

When poor moves are checked first, pruning happens later, so more nodes must be examined.

This shows that branching factor and depth alone cannot fully explain an algorithm's performance. The arrangement of values also matters.

### 3.7 Conclusion

Alpha-Beta Pruning is more efficient than normal Minimax because it avoids checking unnecessary game states.

I would recommend Alpha-Beta Pruning for the game engine because it gives the same decision as Minimax while reducing computation. Good move ordering can improve its performance further.

## TASK 4 – Minimax and the Cost of Examining Every Possibility

### 4.1 Introduction

In a two-player strategy game, the AI must decide its next move by considering the possible moves of the opponent.

Minimax helps the AI select a move by looking ahead several steps.

However, examining every possible move becomes expensive when the number of moves and search depth increase.

### 4.2 Mathematical Calculation

Let:

- b = Average number of legal moves (branching factor).
- d = Search depth.

The number of nodes at each level is:

| Level | Number of nodes |
|---|---|
| 0 | 1 |
| 1 | \(b\) |
| 2 | \(b^2\) |
| 3 | \(b^3\) |
| d | \(b^d\) |

Total nodes:

$$N = 1 + b + b^2 + \cdots + b^d$$

Using the geometric series formula:

$$\boxed{N = \frac{b^{d+1} - 1}{b - 1}}$$

The number of terminal evaluations is:

$$\boxed{b^d}$$

### 4.3 Example Calculation

Assume the game has an average of 4 legal moves per turn.

Here, b=4.

For depth 3:

$$N = 1 + 4 + 16 + 64$$

$$\boxed{N = 85}$$

For depth 5:

$$N = 1 + 4 + 16 + 64 + 256 + 1024$$

$$\boxed{N = 1365}$$

| Depth | Total nodes | Leaf nodes |
|---|---|---|
| 1 | 5 | 4 |
| 2 | 21 | 16 |
| 3 | 85 | 64 |
| 4 | 341 | 256 |
| 5 | 1365 | 1024 |

As the depth increases, the number of positions increases rapidly.

## 4.4 Evaluation Function

When the algorithm reaches the selected search depth, it uses an evaluation function to estimate the strength of the position.

For example:

$$Evaluation = 2A + 3D - 2O$$

Where:

- A = Number of attacking pieces.
- D = Number of defensive pieces.
- O = Number of opponent's attacking pieces.

A higher score represents a better position for MAX.

## 4.5 Python Program

```
def minimax_tree(b, depth):
    nodes = 0
    def search(level, is_max):
        nonlocal nodes
        nodes += 1
        if level == depth:
            return 1
        values = []
        for i in range(b):
            values.append(search(level+1, not is_max))
        if is_max:
            return max(values)
```

```
        return min(values)
    result = search(0, True)
    return result, nodes
b = 4
for depth in [1, 2, 3, 4, 5]:
    result, nodes = minimax_tree(b, depth)
    print("Depth:", depth)
    print("Nodes Examined:", nodes)
```

This program demonstrates the growth in the number of examined positions.

### 4.6 Experimental Analysis

- At lower depths, Minimax works quickly.
- As the depth increases, more positions must be examined.
- Evaluation-function calls also increase.
- Large branching factors make the problem more expensive.

Plain Minimax has:

$$Time = O(b^d)$$

Its recursive memory requirement is approximately:

$$Space = O(bd)$$

for depth-first traversal.

## 4.7 Scalability and Improved Design

Plain Minimax becomes impractical for games with many possible moves.

To improve it, we can use:

1. Alpha-Beta Pruning to skip unnecessary branches.
2. Move ordering to check promising moves first.
3. Depth limits to control computation.
4. Better evaluation functions to estimate positions.

Conclusion: Minimax is useful for selecting moves in small games, but its computation increases rapidly with depth. A combination of depth-limited Minimax and Alpha-Beta Pruning is a better choice for larger games.

## TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle

### 5.1 Introduction

Constraint Satisfaction Problems (CSP) are problems where we must assign values to variables while following certain rules.

For example, in a university examination timetable, two exams attended by the same students should not be scheduled at the same time.

Each exam is treated as a variable, and each available time slot is its domain.

### 5.2 Problem Representation

Consider three examinations:

- E1 – Mathematics
- E2 – Physics
- E3 – Chemistry

Available time slots:

$$D = \{Morning, Afternoon, Night\}$$

Assume that E1, E2 and E3 share students with one another.

Therefore, no two examinations can happen in the same slot.

| Variable | Initial domain |
|---|---|
| E1 | M, A, N |
| E2 | M, A, N |
| E3 | M, A, N |

Constraint:

$$E1 \neq E2, \quad E2 \neq E3, \quad E1 \neq E3$$

## 5.3 Constraint Propagation

Constraint propagation removes values that cannot satisfy the given rules.

For example, if E1 is assigned Morning, Morning must be removed from the domains of E2 and E3.

The updated domains become:

| Variable | Updated domain |
|---|---|
| E1 | Morning |
| E2 | Afternoon, Night |
| E3 | Afternoon, Night |

This reduces the number of possibilities that the algorithm must check.

## 5.4 Working Steps

1. Create variables and their domains.
2. Define the constraints between variables.
3. Check whether each value has support in neighbouring domains.
4. Remove values that violate the constraints.
5. Repeat until no more values can be removed.
6. If any domain becomes empty, the problem has no solution under the current assignments.
7. If the problem is not solved, use backtracking to assign another value.

## 5.5 Python Program – Constraint Propagation and Backtracking

```
variables = ["E1", "E2", "E3"]
domains = {
    "E1": ["M", "A", "N"],
    "E2": ["M", "A", "N"],
    "E3": ["M", "A", "N"]
}
neighbors = {
    "E1": ["E2", "E3"],
    "E2": ["E1", "E3"],
    "E3": ["E1", "E2"]
}
def propagate(domains):
    changed = True
    while changed:
        changed = False
        for x in variables:
            if len(domains[x]) == 1:
                for y in neighbors[x]:
                    if domains[x][0] in domains[y]:
                        if len(domains[y]) == 1:
                            return False
                        domains[y].remove(domains[x][0])
```

```
                changed = True
            if len(domains[y]) == 0:
                return False
    return True
def solve(domains):
    if not propagate(domains):
        return None
    if all(len(domains[x]) == 1 for x in variables):
        return domains
    var = next(x for x in variables if len(domains[x]) > 1)
    for value in domains[var]:
        new_domains = {
            x: domains[x][:] for x in variables
        }
        new_domains[var] = [value]
        result = solve(new_domains)
        if result:
            return result
    return None
answer = solve(domains)
print("Final Timetable:")
if answer:
    for exam in variables:
```

```
        print(exam, ":", answer[exam][0])
else:
    print("No solution found")
```

### 5.6 Test Results

One possible valid timetable is:

| Examination | Time slot |
|---|---|
| E1 | Morning |
| E2 | Afternoon |
| E3 | Night |

No two examinations are scheduled at the same time.

Test without enough time slots: If only two slots are available for three mutually conflicting exams, the algorithm should report that no valid timetable exists.

### 5.7 Why Is Removing an Unsupported Value Safe?

Suppose a value cannot satisfy the constraint with any value in a neighbouring variable's domain.

That means there is no valid assignment using that value.

Removing it does not remove a correct solution because it could never have been part of a valid solution in the first place.

### 5.8 Conclusion

Constraint propagation reduces unnecessary possibilities before backtracking begins. It makes timetable scheduling easier and reduces the number of assignments that must be checked.

Combining constraint propagation with backtracking is useful for solving complex scheduling problems.

## TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth

### 6.1 Introduction

A delivery company must visit several customers and return to the starting depot.

The aim is to find the shortest possible route while visiting every customer exactly once.

This problem is known as the Travelling Salesman Problem (TSP).

Two approaches can be used:

1. Exhaustive Search.
2. Local Search.

### 6.2 Exhaustive Search

Exhaustive Search checks every possible route and selects the route with the lowest total distance.

For example, consider four locations:

A, B, C and D.

The algorithm generates all possible arrangements of the locations and calculates their total distances.

For n locations, fixing the starting depot gives:

$$\boxed{Number\ of\ routes = (n-1)!}$$

Example

For 4 locations:

$$(4-1)! = 3! = 6$$

Therefore, 6 possible tours must be checked.

| Customers | Possible tours |
|---|---|
| 3 | 2 |
| 4 | 6 |
| 5 | 24 |
| 6 | 120 |
| 7 | 720 |
| 8 | 5040 |
| 10 | 362880 |

The number of possible routes increases very quickly.

### 6.3 Python Program – Exhaustive Search

```
from itertools import permutations

distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(distance)

best_cost = float('inf')

best_path = None

for route in permutations(range(1, n)):
    path = (0,) + route + (0,)
```

```
    cost = 0
    for i in range(len(path)-1):
        cost += distance[path[i]][path[i+1]]
    if cost < best_cost:
        best_cost = cost
        best_path = path
print("Best Route:", best_path)
print("Minimum Distance:", best_cost)
```

**6.4 Example Result**

One optimal route is:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$$

Total distance:

$$10 + 25 + 30 + 15 = 80$$

Optimal cost = 80

**6.5 Local Search Approach**

Local Search starts with one possible route and tries to improve it by making small changes.

For example:

- Start with a random route.
- Swap two customer locations.
- Calculate the new distance.
- If the new route is shorter, accept it.
- Continue until no better route is found.

Unlike Exhaustive Search, Local Search does not check every possible route.

**6.6 Python Program – Local Search**

```
import random
distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
def cost(route):
    total = 0
    for i in range(len(route)-1):
        total += distance[route[i]][route[i+1]]
    return total
def local_search():
    route = [0, 1, 2, 3, 0]
    best = cost(route)
    improved = True
    while improved:
        improved = False
        for i in range(1, 3):
            for j in range(i+1, 4):
                new_route = route[:]
```

```
            new_route[i], new_route[j] = (
                new_route[j], new_route[i]
            )
            new_cost = cost(new_route)
            if new_cost < best:
                route = new_route
                best = new_cost
                improved = True
    return route, best
print("Best Route:", local_search()[0])
print("Minimum Distance:", local_search()[1])
```

### 6.7 Exhaustive Search vs Local Search

| Feature | Exhaustive Search | Local Search |
|---|---|---|
| Method | Checks every route | Improves an existing route |
| Time | Very high | Usually lower |
| Optimal solution | Guaranteed | Not guaranteed |
| Memory | Depends on implementation | Low |
| Suitable for | Small problems | Larger problems |

## 6.8 Why Does Computational Effort Increase?

The number of possible tours increases factorially.

For example:

- 5 customers require only 24 tours.
- 8 customers require 5040 tours.
- 10 customers require 362880 tours.
- 12 customers require 39916800 tours.

Even adding a few customers can greatly increase the computation time.

## 6.9 Why Can Local Search Fail?

Local Search may stop at a local minimum.

A local minimum is a solution that cannot be improved by the allowed small changes, even though a better overall solution exists.

It may also get stuck on a plateau where several nearby routes have the same cost.

Using random restarts or simulated annealing can help overcome these problems.

## 6.10 Deployment Decision

I would use Exhaustive Search for small delivery networks because it guarantees the best possible route.

For larger networks, I would use Local Search because it can find a good route with much less computation.

Conclusion: Exhaustive Search is accurate but becomes expensive as the number of customers increases. Local Search is more practical for large delivery networks, although it may not always find the optimal solution.

## TASK 7 – Diagnose, Redesign and Validate the Computational Strategy

### 7.1 Introduction

From the previous tasks, I selected Exhaustive Search in the Travelling Salesman Problem as the algorithm that needs improvement.

The main problem is that Exhaustive Search checks every possible route to find the shortest one.

As the number of customers increases, the number of possible routes becomes extremely large. This makes the algorithm slow and unsuitable for large delivery networks.

### A. Diagnose the Computational Bottleneck

The main limitation of Exhaustive Search is the large number of route calculations.

For n locations, the number of possible tours is:

$$T(n) = (n-1)!$$

For example:

| Locations | Possible tours |
|---|---|
| 4 | 6 |
| 5 | 24 |
| 6 | 120 |
| 8 | 5040 |
| 10 | 362880 |
| 12 | 39916800 |

The algorithm wastes time calculating routes that are already longer than a previously discovered good route.

Main bottleneck: Repeatedly calculating the total distance of unnecessary routes.

**B. Define the Design Objective**

The main objective of my redesigned algorithm is to reduce the number of routes examined without losing the ability to find the optimal solution.

The redesigned algorithm should:

1. Avoid exploring routes that cannot improve the current best solution.
2. Reduce unnecessary distance calculations.
3. Maintain the correct shortest route.
4. Work efficiently for larger delivery networks.

**C. Design the Revised Computational Strategy**

I propose an improved method called Branch and Bound Search.

Instead of checking every possible route, the algorithm calculates a lower bound for each partial route.

If the estimated minimum possible cost is already greater than or equal to the best complete route found so far, that branch is rejected.

Working steps

1. Start from the delivery depot.
2. Select an unvisited customer.
3. Calculate the current route cost.
4. Estimate the minimum possible remaining cost.

5. Compare it with the best route found.
6. If the estimated cost is higher, stop exploring that route.
7. Otherwise, continue to the next customer.
8. Repeat until the best complete route is found.

Pseudocode

START

Set best_cost = infinity

Start from the depot

Generate possible next locations

FOR each location:

Calculate current route cost

Calculate lower bound

IF lower bound >= best_cost:

Skip this route

ELSE:

Continue searching

IF all locations are visited:

Return to depot

IF route cost < best_cost:

Update best route

PRINT best route and minimum cost

STOP

Advantages

- Avoids unnecessary route calculations.
- Can still find the optimal route.
- Works better than basic Exhaustive Search when many branches can be rejected.

Limitation

The method may still take a long time in the worst case because its worst-case complexity remains factorial.

**D. Predict Before Implementing**

I expect the redesigned Branch and Bound method to examine fewer routes than Exhaustive Search.

It should also reduce execution time, especially when a good route is found early.

The main performance measure will be the number of nodes expanded.

**E. Implement and Validate the Redesign**

Python Program – Improved Branch and Bound

```
distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(distance)
best_cost = float('inf')
best_path = []
nodes = 0
def mst(unvisited):
    if len(unvisited) <= 1:
        return 0
    used = {unvisited[0]}
    total = 0
    while len(used) < len(unvisited):
        edge = min(
            (distance[u][v], v)
            for u in used
            for v in unvisited
            if v not in used
```

```
        )
        total += edge[0]
        used.add(edge[1])
    return total
def lower_bound(current, unvisited, cost):
    if not unvisited:
        return cost + distance[current][0]
    first = min(distance[current][x] for x in unvisited)
    last = min(distance[x][0] for x in unvisited)
    return cost + first + mst(unvisited) + last
def search(current, unvisited, path, cost):
    global best_cost, best_path, nodes
    nodes += 1
    bound = lower_bound(current, unvisited, cost)
    if bound >= best_cost:
        return
    if not unvisited:
        total = cost + distance[current][0]
        if total < best_cost:
            best_cost = total
            best_path = path + [0]
        return
    choices = sorted(
```

```
        unvisited,
        key=lambda x: distance[current][x]
    )
    for city in choices:
        search(
            city,
            unvisited - {city},
            path + [city],
            cost + distance[current][city]
        )
search(0, set(range(1, n)), [0], 0)
print("Best Route:", best_path)
print("Minimum Cost:", best_cost)
print("Nodes Expanded:", nodes)
```

**Expected validation**

For the four-location example:

- The original Exhaustive Search checks all 6 tours.
- The improved method uses lower-bound calculations to reject unnecessary branches.
- Both methods should return the same optimal cost of 80.

The exact number of expanded nodes depends on the search order and pruning conditions.

**F. Defend the Final Design**

The redesigned algorithm is better because it avoids exploring branches that cannot produce a better solution.

| Feature | Exhaustive Search | Branch and Bound |
|---|---|---|
| Route checking | All possible routes | Only promising routes |
| Optimal solution | Yes | Yes |
| Unnecessary calculations | More | Fewer when pruning works |
| Worst-case time | O(n!) | O(n!) |
| Practical performance | Poor for large problems | Often better |

Branch and Bound requires additional calculations and memory to maintain the current route and best solution.

However, this extra work is useful when it prevents many unnecessary route calculations.

Conclusion: I would recommend Branch and Bound for the delivery planning problem because it maintains optimality while reducing unnecessary exploration. For very large problems, local search may still be preferred when a quick approximate solution is sufficient.

**Overall Conclusion:**

In this assignment, I studied different AI search and optimization techniques and understood how they can be used to solve real-world problems.

The main findings are:

1. A* Search is suitable for finding efficient routes on campus maps.
2. Online Search is useful when the environment is unknown.
3. Alpha-Beta Pruning reduces unnecessary game-tree exploration.
4. Minimax helps an AI select better moves by examining possible future positions.
5. Constraint Propagation reduces unnecessary assignments in timetable scheduling.
6. Exhaustive Search guarantees the best delivery route but becomes expensive for larger problems.
7. Branch and Bound improves route searching by avoiding unnecessary calculations.

Overall, I understood that selecting an AI algorithm depends on the problem, available information, search space and computational resources. A good algorithm should not only produce the correct answer but should also use time and memory efficiently.